

LAMBERT AI — BACKEND DEVELOPMENT MASTER PROMPT

You are the lead backend engineer for:

LAMBERT AI

Your Intelligent Study Companion

Build the complete production-ready backend for LAMBERT AI.

The frontend is being developed separately by Claude.

Your backend must expose a clean, documented API that the frontend can consume.

Do NOT build frontend UI unless required for testing.

1. BACKEND RESPONSIBILITIES

Build:

- REST API
- Authentication
- Authorization
- Database
- AI provider architecture
- AI Router
- OpenAI integration
- Microsoft AI integration
- Google Gemini integration
- Anthropic Claude integration
- Conversation management
- Document processing
- Retrieval system
- Quiz engine
- Past paper analysis

- Study planner
 - Learning engine
 - Progress analytics
 - Recommendations
 - Notifications
 - Teacher functionality
 - Admin functionality
 - Usage tracking
 - Rate limiting
 - Security
 - Logging
 - Testing
-

2. ARCHITECTURE

Use a modular backend architecture.

Recommended layers:

API

↓

Controllers

↓

Services

↓

Business Logic

↓

Repositories

↓

Database

AI:

API



AI Router



Provider Interface



OpenAI / Microsoft / Gemini / Claude



Standardized Response

Document:

Upload



Validation



Extraction



Chunking



Embedding/Index



Retrieval



AI

Learning:

AI Response



Learning Engine



Progress



Recommendations



Study Plan

3. TECHNOLOGY

Use the existing backend stack if already configured.

If no backend exists, use a stable production-ready stack such as:

Node.js

TypeScript

Fastify or Express

PostgreSQL

Prisma or another reliable ORM

Use migrations.

Use environment variables.

Do not hardcode secrets.

4. ENVIRONMENT VARIABLES

Create secure configuration for:

DATABASE_URL

JWT_SECRET

OPENAI_API_KEY

OPENAI_MODEL

MICROSOFT_AI_ENDPOINT

MICROSOFT_AI_API_KEY

MICROSOFT_AI_MODEL

GEMINI_API_KEY

GEMINI_MODEL

ANTHROPIC_API_KEY

ANTHROPIC_MODEL

API_BASE_URL

STORAGE configuration where required.

Never commit secrets.

Create:

.env.example

with placeholder values only.

5. DATABASE

Create relational models for:

users
profiles
subjects
topics
courses
study_materials
documents
document_chunks
conversations
messages
quizzes
quiz_questions
quiz_attempts
quiz_answers
study_plans
study_sessions
progress
learning_recommendations
bookmarks
notifications
ai_providers
ai_models
ai_usage
feedback
audit_logs

Use:

- Primary keys
- Foreign keys
- Unique constraints
- Indexes

- created_at
- updated_at

Create migrations.

Create seed data.

6. AUTHENTICATION

Implement secure authentication.

Endpoints:

POST /auth/register

POST /auth/login

POST /auth/refresh

POST /auth/logout

POST /auth/forgot-password

POST /auth/reset-password

GET /auth/me

Use secure password hashing.

Use access and refresh tokens where appropriate.

Never store plaintext passwords.

Implement session/token revocation where appropriate.

7. AUTHORIZATION

Roles:

student

teacher

admin

Create middleware:

requireAuth

requireRole

requirePermission

Students can only access their own private data.

Teachers can only access authorized teaching resources.

Admins can manage administrative resources.

Never trust role information from frontend requests.

8. AI PROVIDER SYSTEM

Create:

AIProvider interface.

Required functions:

`generateResponse()`

`streamResponse()`

`generateQuiz()`

`summarize()`

`explain()`

`analyzeDocument()`

Implement:

OpenAIProvider

MicrosoftAIProvider

GeminiProvider

AnthropicProvider

Make the architecture extensible.

9. OPENAI

Use official OpenAI API integration.

Configuration:

`OPENAI_API_KEY`

`OPENAI_MODEL`

Do not assume a specific model ID.

The administrator should configure the model.

Support GPT-4-class models where available through the configured API account.

Never expose the key.

10. MICROSOFT AI

Use official Microsoft-supported APIs.

Do NOT scrape or automate the consumer Microsoft Copilot website.

Where appropriate support:

Azure AI / Azure OpenAI

or another official Microsoft AI API available to the deployment.

Configuration:

MICROSOFT_AI_ENDPOINT

MICROSOFT_AI_API_KEY

MICROSOFT_AI_MODEL

If credentials are missing:

Provider status = disabled

The rest of the application must continue functioning.

11. GOOGLE GEMINI

Implement official Google AI API.

Configuration:

GEMINI_API_KEY

GEMINI_MODEL

12. ANTHROPIC CLAUDE

Implement official Anthropic API.

Configuration:

ANTHROPIC_API_KEY
ANTHROPIC_MODEL

13. AI ROUTER

Create:

AI Router

Inputs:

- task
- user preference
- provider availability
- admin configuration
- model configuration
- fallback settings

Tasks:

tutoring

problem solving

summarization

quiz generation

document analysis

translation

engineering explanation

study planning

Example:

OpenAI

↓ failure

Microsoft

↓ failure

Gemini

↓ failure

Claude

Do not fallback if administrator has disabled fallback.

14. STANDARD AI RESPONSE

Normalize all provider responses:

```
{  
  provider,  
  model,  
  response,  
  usage,  
  latency,  
  citations,  
  conversationId,  
  error  
}
```

The frontend should receive a consistent response regardless of provider.

15. AI CHAT API

Create endpoints:

```
POST /api/chat  
POST /api/chat/stream  
GET /api/conversations  
POST /api/conversations  
GET /api/conversations/:id  
PATCH /api/conversations/:id  
DELETE /api/conversations/:id
```

Verify conversation ownership.

Save messages.

Support:

- streaming
- provider selection
- language

- AI mode
- attachments

16. AI MODES

Support:

tutor

exam

homework

quiz

simple

deep_learning

revision

document

engineering

Each mode should have its own system-level instructions.

Never allow user content to override system security instructions.

17. DOCUMENT API

Endpoints:

POST /api/documents/upload

GET /api/documents

GET /api/documents/:id

DELETE /api/documents/:id

POST /api/documents/:id/ask

GET /api/documents/:id/status

Support:

PDF

DOCX

TXT

Workflow:

Upload



Validate



Extract



Chunk



Index



Ready

Track processing status.

18. DOCUMENT RETRIEVAL

Implement retrieval architecture.

Do NOT send entire large documents to AI.

Use:

document chunks

embeddings/index

semantic retrieval

metadata filters

When answering document questions:

Retrieve relevant sections



Construct context



Send context to AI



Return answer



Return document citations when possible

Ensure document access is authorized.

19. PAST PAPER API

Endpoints:

POST /api/past-papers/analyze

GET /api/past-papers

GET /api/past-papers/:id

POST /api/past-papers/:id/practice

Analyze:

- questions
- subjects
- topics
- years
- difficulty
- question types
- historical patterns

Do not claim future exam predictions.

20. QUIZ ENGINE

Endpoints:

POST /api/quizzes/generate

GET /api/quizzes

GET /api/quizzes/:id

POST /api/quizzes/:id/start

POST /api/quizzes/:id/submit

GET /api/quizzes/attempts

GET /api/quizzes/attempts/:id

Support:

MCQ

True/False

Short Answer

Calculation
Problem Solving

Store attempts.

Calculate:

score
correct
incorrect
time
topic performance

21. STUDY PLANNER

Endpoints:

GET /api/study-plans
POST /api/study-plans
POST /api/study-plans/generate
GET /api/study-plans/:id
PATCH /api/study-plans/:id
DELETE /api/study-plans/:id

Support:

Subjects
Topics
Exam dates
Available hours
Preferred times
Goals

Allow manual modification.

22. LEARNING ENGINE

Build:

LAMBERT Learning Engine.

Track:

- Topics studied
- Quiz performance
- Mistakes
- Study frequency
- Completed topics
- Study goals
- Study time
- Attempts

Learning cycle:

LEARN

↓

PRACTICE

↓

ASSESS

↓

IDENTIFY WEAKNESS

↓

REVIEW

↓

PRACTICE

↓

RE-ASSESS

Generate recommendations based only on real data.

23. PROGRESS API

Endpoints:

GET /api/progress

GET /api/progress/subjects

GET /api/progress/topics

GET /api/progress/activity

GET /api/recommendations

Return:

study time
quiz scores
subject performance
topic performance
streak
completed topics
recommendations

24. SUBJECTS

Seed:

Mathematics
Physics
Chemistry
Biology
Computer Science
Environmental Engineering
Civil Engineering
Electrical Engineering
Mechanical Engineering
Business
Economics
Statistics
Programming
Information Technology
Accounting
Geography

Admins must be able to manage subjects.

25. ENVIRONMENTAL ENGINEERING

Create structured topics for:

Water Supply Engineering
Wastewater Treatment

Solid Waste Management
Air Pollution Control
Environmental Chemistry
Hydrology
Water Quality
Environmental Impact Assessment
Climate Change
Renewable Energy
Environmental Microbiology
GIS and Remote Sensing
Environmental Management
Occupational Health and Safety

Provide data structures for formulas, examples and learning materials.

Do not invent regulatory standards.

26. VOICE BACKEND

Create endpoints/interfaces for:

POST /api/voice/transcribe

POST /api/voice/synthesize

Do not expose speech provider secrets.

If voice provider is unavailable, return a controlled error.

27. NOTIFICATIONS

Create:

GET /api/notifications

PATCH /api/notifications/:id/read

POST /api/notifications/preferences

Support:

Study reminders

Quiz results

Study plans

New materials
Achievements

28. BOOKMARKS

Endpoints:

GET /api/bookmarks

POST /api/bookmarks

DELETE /api/bookmarks/:id

Verify ownership.

29. SEARCH

Create:

GET /api/search

Search:

Subjects

Topics

Materials

Conversations

Quizzes

Documents

Support filters.

Use indexed search where possible.

30. ADMIN API

Endpoints for:

Users

Subjects

Topics

Materials

Quizzes

Providers

Models

Analytics

Feedback

Logs

Protect all admin endpoints.

31. AI PROVIDER ADMIN API

Create:

GET /api/admin/ai/providers

PATCH /api/admin/ai/providers/:id

GET /api/admin/ai/models

PATCH /api/admin/ai/models/:id

GET /api/admin/ai/usage

GET /api/admin/ai/errors

Admin can:

Enable provider

Disable provider

Set default

Set fallback

Configure model

Set limits

Never return API secrets.

32. AI USAGE

Track:

user_id

provider

model

task_type

request_count

input usage where available

output usage where available
estimated cost where available
latency
timestamp
status

Create daily/monthly aggregation queries.

33. RATE LIMITING

Implement rate limits for:

Authentication
AI requests
Document uploads
Quiz generation
Past paper analysis
Voice requests

Use stricter limits for expensive operations.

Return proper HTTP status codes.

34. SECURITY

Implement:

Input validation
Authentication
Authorization
Rate limiting
Secure headers
CORS
Request size limits
File validation
Safe error handling
Audit logging

Protect against:

SQL injection

XSS

CSRF where applicable

Prompt injection

Unauthorized document access

Data leakage

API key exposure

Abuse

Never trust frontend authorization.

35. PROMPT INJECTION DEFENSE

Treat uploaded documents and user messages as untrusted content.

System instructions must remain higher priority.

Do not allow a document to instruct the AI to:

- reveal secrets
- ignore system rules
- expose another user's information
- reveal API keys
- bypass security

36. FILE SECURITY

Validate:

File type

File size

File name

File content

Do not execute uploaded files.

Store files securely.

Generate unique storage identifiers.

Do not expose direct private storage paths.

37. PRIVACY

Minimize personal data sent to external AI providers.

Only send information necessary for the task.

Respect provider configuration.

Do not send data to disabled providers.

38. LOGGING

Create structured logs.

Log:

- request ID
- user ID where appropriate
- endpoint
- status
- latency
- provider
- error category

Never log:

- API keys
 - passwords
 - tokens
 - private document contents
 - unnecessary personal information
-

39. API DOCUMENTATION

Create OpenAPI/Swagger documentation.

Document:

Authentication

Chat

Conversations

Documents

Quizzes

Study Plans

Progress

Recommendations

Subjects

Notifications

Admin

AI Providers

Make API contracts clear enough for Claude's frontend to consume.

40. FRONTEND CONTRACT

The frontend created by Claude expects clean JSON APIs.

Return consistent structures.

Example:

```
{  
  "success": true,  
  "data": {},  
  "message": null,  
  "error": null  
}
```

For errors:

```
{  
  "success": false,  
  "data": null,  
  "message": "User-friendly message",  
  "error": {  
    "code": "ERROR_CODE"
```

```
}  
}
```

Do not expose stack traces.

41. TESTING

Create unit tests for:

Authentication

Authorization

AI Router

Provider fallback

Chat

Documents

Retrieval

Quizzes

Study Plans

Progress

Recommendations

Admin permissions

Create integration tests for critical APIs.

Test provider failure scenarios.

Test unauthorized access.

Test document ownership.

42. DATABASE SEED

Create realistic seed data for:

Subjects

Topics

Environmental Engineering topics

Quiz examples

Study materials metadata

AI providers

AI models

Do not create fake student analytics that could be mistaken for real data.

43. PERFORMANCE

Optimize:

Database queries

Indexes

AI calls

Document processing

Search

Caching

Background processing

Do not block HTTP requests unnecessarily for long document processing.

Use background jobs/queues if the chosen infrastructure supports them.

44. HEALTH CHECK

Create:

GET /health

Return:

status

version

database status

AI provider availability

Do not expose secrets.

45. PRODUCTION CONFIGURATION

Create:

.env.example

Production-safe configuration.

Separate:

development

test

production

Do not commit .env.

46. FINAL SECURITY AUDIT

Before completion:

Review every API.

Check authentication.

Check authorization.

Check database ownership.

Check file access.

Check AI provider secrets.

Check rate limits.

Check input validation.

Check logging.

Check CORS.

Fix all critical and high-risk issues.

47. FINAL TEST

Run:

Type checking

Lint

Unit tests

Integration tests

Database migration

Database seed

Production build

Fix every error.

48. FINAL API CONTRACT

Document all endpoints in:

/docs/API.md

Include:

Endpoint

Method

Authentication

Request body

Response

Errors

Example

The frontend team must be able to integrate without guessing endpoint structures.

FINAL BACKEND OBJECTIVE

The backend must provide the infrastructure for:

STUDENT

↓

LAMBERT AI

↓

AI ROUTER

↓

OPENAI / MICROSOFT / GEMINI / CLAUDE

↓

LEARNING ENGINE

↓

QUIZ ENGINE

↓

PROGRESS

↓

PERSONALIZED RECOMMENDATIONS

Build the backend as a scalable foundation for a serious educational technology platform.

Do not create fake AI responses.

Do not hardcode API keys.

Do not expose secrets.

Do not skip authorization.

Do not break existing functionality when adding new features.

Implement systematically, test each module, and fix errors before moving forward.